

R / tidyverse

Data Manipulation Cheat Sheet

read_csv · filter · arrange · slice · select · rename · relocate · mutate · summarise · group_by

Dataset used in all examples:

```
library(tidyverse)
students <- read_csv("students.csv")
```

students.csv contains:

name	(chr)	student name
age	(dbl)	age in years
grade	(chr)	A / B / C / D / F
score	(dbl)	exam score 0-100
country	(chr)	home country
year	(dbl)	study year 1-4

read_csv() — Import a CSV file into R as a tibble *[readr / tidyverse]*

Purpose

Reads a comma-separated values file from disk (or a URL) into an R `tibble` — the tidyverse version of a data frame.

When to use it

Every time you start an analysis and need to load data stored as a .csv file.

Key behaviour

- Automatically detects column types
- Reads header row as column names
- Faster than base R `read.csv()`

Syntax

```
read_csv(file, col_types = NULL,
          skip = 0, na = c('', 'NA'))
```

Basic example

```
students <- read_csv('students.csv')
```

With options

```
students <- read_csv(
  'students.csv',
  col_types = cols(
    score = col_double(),
    grade = col_factor()
  ),
  na = c('', 'NA', 'N/A')
)
```

From a URL

```
df <- read_csv("https://example.com/data.csv")
```

Argument	Type	Description
file	string	Path to the CSV file, or a URL
col_types	cols()	Override auto-detected column types
skip	integer	Number of rows to skip before the header
na	vector	Strings to treat as missing (NA)

filter() — Keep rows that match a condition *[dplyr / tidyverse]*

Purpose

Returns only the rows where the logical condition(s) evaluate to TRUE. All other rows are dropped.

Operators

Single condition

```
students |> filter(grade == 'A')
```

Multiple conditions — AND

```
students |> filter(grade == 'A', score > 90)
```

```

== equal          != not equal >
greater          < less >=
greater/eq       <= less/eq %in%
in vector is.na() missing

```

Combining conditions

, or & → AND (both must be TRUE) |
 → OR (either must be TRUE) ! →
 NOT (negates condition)

OR condition

```
students |> filter(grade == 'A' | grade == 'B')
```

Match any value in a list

```
students |> filter(country %in% c('Hungary', 'Germany'))
```

Exclude missing values

```
students |> filter(!is.na(score))
```

Range

```
students |> filter(score >= 70, score <= 90)
```

💡 The comma between conditions means AND — both must be true. Use | explicitly for OR logic.

arrange() — Sort rows by one or more columns [dplyr / tidyverse]

Purpose

Reorders the rows of a data frame based on the values in one or more columns. Does not change which rows are present — only their order.

Default order

Ascending (smallest → largest). Wrap in desc() for descending.

Multiple columns

When you supply multiple columns, ties in the first column are broken by the second column, and so on.

Ascending (default)

```
students |> arrange(score)
```

Descending

```
students |> arrange(desc(score))
```

Multiple columns

```
# Sort by grade A→F, then by score high→low
students |> arrange(grade, desc(score))
```

Combined with filter

```
students |>
  filter(grade == 'A') |>
  arrange(desc(score))
```

💡 arrange() does not permanently sort the data — the order resets if you filter or join afterward. Use it as the last step before displaying or exporting.

slice_max() / slice_min() / slice_head() / slice_tail() — Select rows by position or rank [dplyr / tidyverse]

Purpose

The slice_ family extracts rows based on their rank or physical position — without needing to arrange() first.

Function overview

```

slice_max()  top n rows by value
slice_min()  bottom n rows by value
slice_head() first n rows
slice_tail() last n rows
slice_sample() random n rows

```

Top 3 highest scores

```
students |> slice_max(order_by = score, n = 3)
```

Bottom 3 scores (no ties)

```
students |> slice_min(order_by = score, n = 3,
                      with_ties = FALSE)
```

First 5 rows

```
students |> slice_head(n = 5)
```

Last 5 rows

```
students |> slice_tail(n = 5)
```

	Random sample of 10 <pre>set.seed(42) # for reproducibility students > slice_sample(n = 10)</pre> Top scorer per country (grouped) <pre>students > group_by(country) > slice_max(order_by = score, n = 1) > ungroup()</pre>
--	--

Argument	Type	Description
order_by	column	Column to use for ranking (slice_max / slice_min only)
n	integer	Number of rows to return
with_ties	logical	If FALSE, return exactly n rows even when tied (default TRUE)
prop	double	Return a proportion instead of a fixed count, e.g. prop = 0.1 = 10%

select() — Keep or drop columns [dplyr / tidyverse]

Purpose

Subsets the columns of a data frame. Does not affect rows. Use it early in a pipeline to reduce clutter and memory use.

Selection helpers

starts_with('x') columns starting with x
ends_with('x') columns ending with x
contains('x') columns containing x
everything() all remaining columns
where(is.numeric) columns of a given type

Keep specific columns

```
students |> select(name, score, grade)
```

Drop specific columns (minus sign)

```
students |> select(-year, -country)
```

Using helpers

```
students |> select(name, starts_with('s'))
# keeps: name, score
```

Keep all numeric columns

```
students |> select(where(is.numeric))
```

Reorder columns while keeping all

```
# Put name first, then everything else
students |> select(name, everything())
```

💡 select() with a minus sign drops columns; without it, you are listing columns to keep. You cannot mix keep and drop in one call (except with everything()).

rename() — Rename columns without dropping any [dplyr / tidyverse]

Purpose

Changes the name of one or more columns. All other columns are kept unchanged — unlike select(), which would drop unmentioned columns.

Syntax rule

new_name = old_name The new name is always on the LEFT. This is easy to mix

Rename one column

```
students |> rename(exam_score = score)
```

Rename multiple columns

```
students |> rename(
  exam_score = score,
  study_year = year,
  home_country = country
)
```

up — if you get an error, check you haven't reversed it.

rename_with()

Use `rename_with()` to apply a function to column names, e.g. to convert all names to uppercase or lowercase at once.

Rename + keep specific columns with select()

```
# select() can also rename while keeping
students |> select(name, exam_score = score)
```

Rename all columns to lowercase

```
students |> rename_with(tolower)
```

Replace spaces with underscores

```
students |>
  rename_with(~ gsub(' ', '_', .x))
```

💡 Remember: `new_name = old_name`. Think of it as 'I want to call the old column by a new name'.

relocate() — Move columns to a new position *[dplyr / tidyverse]*

Purpose

Moves one or more columns to a different position in the data frame without dropping or renaming anything. Useful for making important columns visible on the left.

Position helpers

`.before = col` place before this column
`.after = col` place after this column (neither) move to position 1 (far left)

Move to the front

```
students |> relocate(score)
# score is now column 1
```

Move before a specific column

```
students |> relocate(score, .before = name)
```

Move after a specific column

```
students |> relocate(country, .after = name)
```

Move multiple columns at once

```
students |>
  relocate(score, grade, .after = name)
```

Use after mutate() to bring new columns forward

```
students |>
  mutate(passed = score >= 50) |>
  relocate(passed, .after = name)
```

💡 `relocate()` is purely cosmetic — it helps readability but has no effect on calculations. Use it as the last step before printing or exporting.

mutate() — Create or modify columns *[dplyr / tidyverse]*

Purpose

Adds new columns or overwrites existing ones. Operates row by row — each row gets its own computed value based on the formula you provide.

Common operations

Math: `score * 2, log(score)`
Logical: `score > 50` →
TRUE/FALSE String: `paste(),`
`str_to_upper()` Conditional: `if_else(),`
`case_when()`

if_else vs case_when

Math transformation

```
students |> mutate(score_pct = score / 100)
```

Boolean column

```
students |> mutate(passed = score >= 50)
```

Conditional with if_else()

```
students |>
  mutate(result = if_else(score >= 50,
                           'Pass', 'Fail'))
```

Multiple categories with case_when()

`if_else()` two outcomes (T/F)
`case_when()` multiple conditions

```
students |>
  mutate(level = case_when(
    score >= 90 ~ 'Excellent',
    score >= 70 ~ 'Good',
    score >= 50 ~ 'Pass',
    TRUE      ~ 'Fail'      # default
  ))
```

Multiple mutations in one call

```
students |>
  mutate(
    score_pct = round(score / 100, 2),
    passed    = score >= 50,
    age_group = if_else(age < 22, 'Junior',
                        'Senior')
  )
```

💡 `mutate()` never changes the number of rows — it only adds or updates columns. To reduce rows, use `filter()` or `summarise()`.

`summarise()` — Collapse rows into summary statistics *[dplyr / tidyverse]*

Purpose

Reduces many rows to one (or one per group). Use it to compute means, counts, totals, or any aggregate statistic.

Common summary functions

`n()` count rows `sum()`
total `mean()` average `median()`
middle value `sd()` standard
deviation `min()` smallest value
`max()` largest value
`n_distinct()` unique count

Always add `na.rm = TRUE`

If any value in the column is NA, `mean()` etc. return NA unless you tell R to ignore missing values.

Single statistic

```
students |> summarise(mean_score =
  mean(score))
```

Multiple statistics at once

```
students |>
  summarise(
    n          = n(),
    mean_score = mean(score, na.rm = TRUE),
    sd_score   = sd(score,   na.rm = TRUE),
    min_score  = min(score,  na.rm = TRUE),
    max_score  = max(score,  na.rm = TRUE)
  )
```

Count distinct values

```
students |>
  summarise(
    n_countries = n_distinct(country),
    n_students  = n()
  )
```

Proportion (TRUE = 1, FALSE = 0)

```
students |>
  summarise(pct_passed = mean(score >= 50) *
    100)
```

💡 `summarise()` alone collapses everything to one row. Pair it with `group_by()` (see below) to get one row per group — this is the most common use case.

`group_by` + `summarise()` — Split-apply-combine: statistics by group *[dplyr / tidyverse]*

Purpose

Mean score by grade

```
students |>
```

`group_by()` marks invisible group boundaries in the data. Every subsequent `summarise()` (or `mutate()`) then runs independently within each group and combines the results.

The pattern

1. Split `group_by()` 2. Apply `summarise()` 3. Combine automatic

Always use `.groups = 'drop'`

After summarising multiple groups, the result stays partially grouped. Setting `.groups = 'drop'` removes all grouping, giving you a clean standard data frame.

Multiple group variables

You can group by multiple columns. You will get one output row for every unique combination.

```
group_by(grade) |>
  summarise(mean_score = mean(score, na.rm =
    TRUE),
            n           = n(),
            .groups     = 'drop')
```

Multiple statistics per group

```
students |>
  group_by(country) |>
  summarise(
    n_students = n(),
    mean_score = round(mean(score, na.rm =
      TRUE), 1),
    pass_rate  = mean(score >= 50, na.rm =
      TRUE) * 100,
    .groups    = 'drop'
  ) |>
  arrange(desc(mean_score))
```

Two grouping variables

```
students |>
  group_by(country, grade) |>
  summarise(n = n(), .groups = 'drop')
```

`group_by()` + `mutate()` — add group stats as new column

```
# Add country mean without collapsing rows
students |>
  group_by(country) |>
  mutate(country_mean = mean(score, na.rm =
    TRUE)) |>
  ungroup()
```

💡 `group_by()` + `mutate()` keeps all rows but adds group-level values to each row. `group_by()` + `summarise()` collapses to one row per group. Both patterns are extremely common.

Quick Reference

Function	Package	What it does	Minimal example
<code>read_csv()</code>	readr	Import CSV file	<code>df <- read_csv("file.csv")</code>
<code>filter()</code>	dplyr	Keep matching rows	<code>df > filter(x > 5)</code>
<code>arrange()</code>	dplyr	Sort rows	<code>df > arrange(desc(x))</code>
<code>slice_max()</code>	dplyr	Top n rows by value	<code>df > slice_max(x, n = 3)</code>
<code>slice_min()</code>	dplyr	Bottom n rows by value	<code>df > slice_min(x, n = 3)</code>
<code>slice_head()</code>	dplyr	First n rows	<code>df > slice_head(n = 5)</code>
<code>slice_tail()</code>	dplyr	Last n rows	<code>df > slice_tail(n = 5)</code>
<code>select()</code>	dplyr	Keep/drop columns	<code>df > select(a, b, -c)</code>
<code>rename()</code>	dplyr	Rename columns	<code>df > rename(new = old)</code>
<code>relocate()</code>	dplyr	Move columns	<code>df > relocate(x, .before = y)</code>
<code>mutate()</code>	dplyr	Create/modify columns	<code>df > mutate(z = x + y)</code>
<code>summarise()</code>	dplyr	Collapse to summary row(s)	<code>df > summarise(m = mean(x))</code>

<code>group_by()</code>	<code>dplyr</code>	Group for subsequent operations	<code>df > group_by(cat) > summarise(...)</code>
-------------------------	--------------------	---------------------------------	--